

SkillForge SQL & Database Mastery

100 Most Common & Important SQL / Database Interview Questions with Answers

Set 1: Beginner + Intermediate — 60 Questions

This set covers fundamentals beyond basic syntax and includes RDBMS, SQL, keys, joins, normalization, transactions, and common interview concepts.

1. What is a Database?

A database is an organized collection of data stored and accessed electronically. It allows efficient storage, retrieval, and management of data.

2. What is DBMS?

DBMS (Database Management System) is software that interacts with users, applications, and the database itself to capture and analyze data. Examples: MySQL, PostgreSQL, Oracle, SQL Server.

3. What is RDBMS?

RDBMS (Relational DBMS) stores data in tables with rows and columns and enforces relationships via keys and constraints. It is based on relational model and uses SQL.

4. What is SQL?

SQL (Structured Query Language) is the standard language for managing and querying relational databases. It is used to create, read, update, and delete data.

5. What are the types of SQL commands?

DDL (CREATE, ALTER, DROP, TRUNCATE), DML (SELECT, INSERT, UPDATE, DELETE), DCL (GRANT, REVOKE), TCL (COMMIT, ROLLBACK, SAVEPOINT).

6. What is a Table?

A table is a collection of rows and columns that stores data in a structured format. Each row is a record and each column is an attribute.

7. What is a Primary Key?

Primary Key uniquely identifies each row in a table. It cannot be NULL and only one primary key per table. It is usually clustered indexed.

8. What is a Foreign Key?

A Foreign Key is a field that references the primary key of another table. It enforces referential integrity and prevents orphan records.

9. What is a Unique Key?

Unique Key ensures all values in a column are distinct but allows one NULL (in SQL Server) or multiple NULLs (in PostgreSQL/MySQL). Multiple unique keys per table are allowed.

10. Difference between Primary Key and Unique Key?

Primary Key = UNIQUE + NOT NULL, one per table. Unique Key may allow NULLs and multiple per table. Primary Key is usually clustered index by default.

11. What is a Candidate Key?

A candidate key is a minimal superkey that could qualify as a primary key. A table can have multiple candidate keys, one is chosen as primary.

12. What is a Composite Key?

A composite key is a primary key consisting of two or more columns combined to uniquely identify a row.

13. What is Normalization?

Normalization is the process of organizing data to reduce redundancy and improve integrity by dividing large tables into smaller related tables.

14. What are 1NF, 2NF, 3NF, BCNF?

1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependency on composite key. 3NF: 2NF + no transitive dependency. BCNF: stricter 3NF where every determinant is a superkey.

15. What is Denormalization?

Denormalization adds redundancy back for performance by combining tables or adding duplicate columns. Used in OLAP, reporting, and read-heavy systems.

16. What is an Index? Why use it?

An index is a B-Tree (or other) structure that speeds up data retrieval without scanning full table. It improves SELECT but slows INSERT/UPDATE/DELETE.

17. Types of Indexes?

Clustered: data sorted and stored with index (one per table). Non-clustered: separate structure pointing to data. Unique, Composite, Full-text, Bitmap, etc.

18. What is a View?

A View is a virtual table based on result of a SELECT query. It does not store data physically (unless materialized) and simplifies complex queries and security.

19. View vs Table?

Table stores physical data, View is saved query. Table can be indexed and modified directly, View is usually read-only unless updatable view with conditions.

20. What is a Materialized View?

A materialized view stores query result physically and needs refresh. It is faster for heavy aggregations and used in reporting/data warehouses.

21. What is a JOIN? Types of JOINS?

JOIN combines rows from two tables based on related column. Types: INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS, SELF.

22. Difference between INNER JOIN and OUTER JOIN?

INNER returns only matching rows from both tables. OUTER returns matching plus non-matching rows with NULLs – LEFT keeps all left rows, RIGHT keeps all right, FULL keeps both.

23. What is a Self Join?

A Self Join is a table joining to itself using aliases. Used for hierarchical data like employee-manager relationship: `SELECT e.name, m.name FROM emp e JOIN emp m ON e.manager_id = m.id.`

24. What is a Cross Join?

Cross Join returns Cartesian product of both tables – every row of first table combined with every row of second. If 10 rows x 20 rows = 200 rows.

25. What is UNION and UNION ALL?

UNION combines result sets of two SELECTs and removes duplicates (does DISTINCT sort). UNION ALL keeps duplicates and is faster.

26. Difference between UNION and JOIN?

UNION stacks rows vertically (same number of columns), JOIN combines columns horizontally based on condition.

27. What is GROUP BY?

GROUP BY groups rows with same values to allow aggregate functions like COUNT, SUM, AVG, MIN, MAX per group.

28. Difference between WHERE and HAVING?

WHERE filters rows before grouping/aggregation. HAVING filters groups after GROUP BY. WHERE cannot use aggregates, HAVING can.

29. What is ORDER BY?

ORDER BY sorts result set ascending (ASC) or descending (DESC). It can sort by column name, expression, or positional number.

30. What is DISTINCT?

DISTINCT removes duplicate rows from result set. SELECT DISTINCT dept removes duplicate department names.

31. Difference between DELETE, TRUNCATE, and DROP?

DELETE is DML, row-by-row logged, can have WHERE and be rolled back. TRUNCATE is DDL, deallocates pages, fast, resets identity. DROP removes table structure entirely.

32. What is NULL? How to check for NULL?

NULL means unknown/missing value, not equal to anything. Check with IS NULL or IS NOT NULL, not = NULL. COUNT(*) counts NULLs, COUNT(col) ignores NULLs.

33. What is COALESCE() and IFNULL()/NVL()?

COALESCE returns first non-NULL value from list: COALESCE(a,b,c). IFNULL (MySQL)/NVL (Oracle)/ISNULL (SQL Server) does same for two args. Used to handle NULLs.

34. What is a Constraint? Types?

Constraints enforce rules: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK (e.g., age > 18), DEFAULT. They ensure data integrity.

35. What is Referential Integrity?

Referential integrity ensures foreign key value always references existing primary key value or is NULL. Prevents orphan records via FOREIGN KEY constraint.

36. What is ACID?

ACID for transactions: Atomicity (all or nothing), Consistency (valid state), Isolation (concurrent tx don't interfere), Durability (committed data survives crash via WAL).

37. What is a Transaction?

A transaction is a unit of work with one or more SQL statements that must succeed or fail together. Example: money transfer debit + credit.

38. What are TCL commands?

COMMIT saves transaction, ROLLBACK undoes, SAVEPOINT creates intermediate point to rollback partially.

39. What is a Stored Procedure?

A stored procedure is a precompiled group of SQL statements stored in DB and executed via CALL/EXEC. Improves performance, reusability, and security.

40. Stored Procedure vs Function?

Procedure may not return value, can have OUT params, can do DML, called via EXEC. Function must return value, can be used in SELECT, usually no DML, deterministic preferred.

41. What is a Trigger?

A trigger is code that automatically fires on INSERT/UPDATE/DELETE event. Used for auditing, validation, but can hurt performance if overused.

42. What is a Cursor?

A cursor is used to process rows one-by-one from result set. It is row-by-row processing and generally slower than set-based operations – avoid if possible.

43. Difference between CHAR and VARCHAR?

CHAR(n) is fixed length, pads spaces, faster for short fixed data. VARCHAR(n) is variable length, stores only actual length + overhead, saves space.

44. Difference between VARCHAR and TEXT?

VARCHAR has length limit (e.g., 255/65535) and can be indexed fully. TEXT is for large strings (up to 64KB-4GB), stored out-of-line (TOAST), not fully indexable by default.

45. What is an Aggregate Function?

Aggregate performs calculation on set of rows: COUNT, SUM, AVG, MIN, MAX. Usually used with GROUP BY.

46. What are String Functions?

Functions to manipulate strings: UPPER, LOWER, SUBSTRING, LENGTH, TRIM, CONCAT, REPLACE, LIKE, POSITION. Vary slightly by DB.

47. What is a Subquery?

A subquery is a query inside another query – in WHERE, FROM, or SELECT. Types: scalar (one value), row, table. Example: WHERE salary > (SELECT AVG(salary) FROM emp).

48. Correlated vs Non-Correlated Subquery?

Non-correlated executes once independently. Correlated references outer query and executes per row – e.g., WHERE EXISTS (SELECT 1 FROM dept d WHERE d.id = e.dept_id) – slower, often replaced by JOIN.

49. What is a CTE (Common Table Expression)?

CTE is temporary named result set defined with WITH clause. Improves readability and allows recursive queries for hierarchies. Example: WITH cte AS (SELECT...) SELECT * FROM cte.

50. What is N+1 Query Problem?

N+1 occurs when 1 query fetches parent and N queries fetch children per parent (ORM lazy loading). Fix with JOIN FETCH, batch loading, or IN clause.

51. What is an ER Diagram?

ER (Entity-Relationship) diagram shows entities, attributes, and relationships. Used for database design before creating tables.

52. What is Cardinality?

Cardinality defines relationship count: One-to-One, One-to-Many, Many-to-Many. High cardinality column has many distinct values (good for index).

53. What is a Relationship? Types?

Relationship is association between tables via keys. Types: One-to-One (user-profile), One-to-Many (dept-employees), Many-to-Many (student-course via junction table).

54. What is Data Integrity?

Data Integrity ensures accuracy and consistency: Entity (PK), Referential (FK), Domain (CHECK, data type), User-defined integrity.

55. Difference between OLTP and OLAP?

OLTP is row-store, highly transactional, many small writes, normalized. OLAP is columnar, analytical, few large reads, denormalized/star schema, optimized for aggregation.

56. What is a Data Warehouse?

Data Warehouse is central repository for analytics from multiple sources, optimized for OLAP, uses ETL, star/snowflake schema, e.g., Snowflake, BigQuery, Redshift.

57. What is a Schema?

Schema is logical container for tables, views, indexes, procedures. Also means structure of database – definition of tables, columns, types, relationships.

58. Difference between WHERE clause and ON clause in JOINS?

ON defines join condition and for OUTER JOIN affects which rows preserved. WHERE filters after join. For INNER they are similar, for LEFT JOIN ON keeps non-matching left rows, WHERE may remove them.

59. What is LIKE and Wildcards?

LIKE searches pattern: % = zero or more chars, _ = single char. Example: WHERE name LIKE 'A%' starts with A. ILIKE is case-insensitive in PostgreSQL. Use index with prefix search only.

60. What is LIMIT/OFFSET and TOP/FETCH?

LIMIT 10 OFFSET 20 (PostgreSQL/MySQL) or TOP 10 (SQL Server) or FETCH NEXT 10 ROWS restricts result. OFFSET is inefficient for deep pagination – use keyset pagination instead.

Set 2: Professional / 10+ Years Experience — 40 Questions

This section focuses on storage engines, indexing internals, concurrency, MVCC, performance tuning, replication, sharding, distributed transactions and senior engineering judgment.

1. Explain B+ Tree vs Hash vs LSM Tree.

B+ Tree: balanced tree, good for range/equality, read-optimized, in-place updates, used in InnoDB/PostgreSQL.
Hash: O(1) equality only, no range. LSM: write-optimized, buffers in memtable then SSTables with compaction – used in RocksDB, Cassandra, high write throughput.

2. Clustered vs Non-Clustered index internally?

Clustered index stores table data in leaf nodes sorted by key – one per table. Non-clustered stores separate B-Tree with key + row pointer (or PK). Non-clustered needs bookmark lookup unless covering.

3. What is covering index and INCLUDE?

Covering index includes all columns query needs, so engine does index-only scan without table lookup. In SQL Server/PostgreSQL use INCLUDE(col) for non-key columns to cover without bloating tree.

4. What is index selectivity and cardinality?

Selectivity = distinct values / total rows. High selectivity (e.g., email) good for index. Low selectivity (e.g., gender) often leads to seq scan. Cardinality estimation drives optimizer plan choice.

5. What is execution plan? How to read EXPLAIN?

Execution plan shows how DB will execute query: scan type (Seq Scan, Index Seek), join type, estimated rows, cost, buffers. In PostgreSQL use EXPLAIN ANALYZE to see actual time/rows vs estimates.

6. Nested Loop vs Hash Join vs Merge Join?

Nested Loop: for small outer + indexed inner lookup. Hash Join: builds hash table for large equi-joins without index. Merge Join: for sorted inputs, efficient for large sorted joins. Optimizer chooses based on size, index, sort.

7. What is SARGable query?

SARGable = Search ARGument able – predicate that can use index. WHERE YEAR(date)=2024 is non-SARGable (function on column). WHERE date >= '2024-01-01' AND date < '2025-01-01' is SARGable and allows index seek.

8. What is parameter sniffing and how to fix?

Optimizer compiles plan based on first parameter value – may be bad for other values. Fixes: OPTION(RECOMPILE), OPTIMIZE FOR UNKNOWN, local variables, plan guides, or rewrite with dynamic SQL.

9. Explain ACID implementation: WAL, redo/undo.

Atomicity/Durability via WAL/redo log flushed before commit. Consistency via constraints. Isolation via locks/MVCC. Undo log for rollback. WAL ensures crash recovery.

10. What are isolation levels and phenomena?

Levels: READ UNCOMMITTED (dirty read), READ COMMITTED (prevents dirty, default Postgres/SQL Server), REPEATABLE READ (prevents non-repeatable), SERIALIZABLE (prevents phantom/write skew). Phenomena: dirty, non-repeatable, phantom, write skew.

11. MVCC – how it works in PostgreSQL/MySQL?

MVCC keeps multiple versions with xmin/xmax transaction IDs. Readers see snapshot without blocking writers. In PostgreSQL dead tuples need VACUUM. In InnoDB uses undo logs for versioning.

12. What is deadlock? How to detect and prevent?

Deadlock when two transactions wait for each other's locks forever. DB detects via wait-for graph and aborts victim with lowest rollback cost. Prevention: consistent lock ordering, reduce lock scope, avoid nested locks, use NOWAIT/SKIP LOCKED.

13. What is lock escalation? Row, Page, Table locks?

Many row locks consume memory, DB escalates to page/table lock to save memory but reduces concurrency. SQL Server escalates after ~5000 locks. Mitigation: batch operations into smaller transactions.

14. Optimistic vs Pessimistic locking?

Pessimistic locks upfront with SELECT FOR UPDATE – prevents conflict but reduces concurrency. Optimistic checks at commit via version column WHERE version=:old – good for low contention, retry on failure.

15. What is SELECT ... FOR UPDATE and SKIP LOCKED?

FOR UPDATE acquires exclusive row locks till commit – for queue/concurrency control. SKIP LOCKED skips already locked rows, allowing multiple workers to dequeue without blocking – used in job queues.

16. How does partitioning work? Types?

Partitioning splits large table into smaller physical pieces: Range (by date), List (by region), Hash (by key), Composite (range+hash). Benefits: partition pruning, faster maintenance, improved manageability.

17. Sharding vs Partitioning? Strategies?

Partitioning within single node. Sharding across nodes/servers. Sharding strategies: key-based, range, hash, consistent hashing (minimal remapping). Challenges: distributed joins, cross-shard transactions, hot shards.

18. What is replication? Lag and handling?

Replication copies data to replicas: async (lag), sync (no lag but slower). Master-Slave for read scaling, Master-Master for write scaling. Handle lag with read-your-writes routing, session stickiness, or read from primary for critical reads.

19. CAP theorem in context of databases?

CAP: during network partition, choose Consistency (CP – return error) or Availability (AP – return stale). RDBMS often CP, NoSQL often AP. In practice tune per use case with eventual consistency.

20. Eventual vs Strong consistency?

Strong: read sees last write immediately (linearizable/quorum). Eventual: updates propagate async, reads may be stale but converge – used in Dynamo, Cassandra for high availability. Choose based on business need.

21. How to diagnose and fix slow queries?

Steps: EXPLAIN ANALYZE, check seq scan vs index seek, check buffers/hit ratio, use pg_stat_statements ORDER BY total_time, slow query log, add covering/partial index, rewrite SARGable, avoid SELECT *, check statistics with ANALYZE.

22. What is query optimizer and cost-based optimization?

Optimizer parses SQL, considers many plans using statistics (n_distinct, histograms, correlation) and cost model (seq_page_cost, random_page_cost, cpu costs). Chooses cheapest plan. Bad stats lead to bad plans.

23. What is cardinality estimation failure?

When optimizer misestimates row count – e.g., correlated columns, skewed data, functions on columns. Leads to wrong join order and memory grant issues. Fix with extended statistics, re-analyze, or query hints.

24. What is filtered/partial index and when to use?

Index with WHERE clause: CREATE INDEX ON orders(status) WHERE status='PENDING'. Smaller, faster, less maintenance. Ideal for queries on subset with low percentage of rows.

25. What is BRIN, GIN, GiST, Full-Text indexes?

BRIN: stores min/max per block range – tiny index for huge naturally ordered tables (time-series). B-Tree: equality/range. GIN: inverted index for jsonb, arrays, full-text. GiST: for range, geometry, KNN. Choose based on data type/query.

26. What is WAL and checkpointing?

WAL (Write-Ahead Log) logs changes before data pages modified. Checkpoint flushes dirty pages from buffer pool/shared_buffers to data files and records LSN. Long checkpoint = faster but longer recovery.

27. How does VACUUM work in PostgreSQL?

PostgreSQL MVCC leaves dead tuples after UPDATE/DELETE. VACUUM marks space reusable, freezes XIDs to prevent wraparound, updates visibility map. Autovacuum handles it, but large bloat needs VACUUM FULL or pg_repack.

28. How to design zero-downtime schema migration?

Use expand/contract pattern: 1) Add new column nullable or new table, 2) Backfill in small batches, 3) Dual-write, 4) Create index CONCURRENTLY, 5) Add constraint NOT VALID then VALIDATE, 6) Swap via view or rename. Avoid long exclusive locks.

29. How to handle hot partitions / hot keys?

Hot partition when one partition gets disproportionate traffic. Fix: add random suffix/prefix to key (write sharding), use composite key with high cardinality, use consistent hashing, adaptive capacity, or split hot key via application-level sharding.

30. What is connection pooling? Transaction vs Session?

Pooling reuses connections to avoid overhead. PgBouncer transaction pooling reuses connection per transaction – fastest but cannot use session features (prepared statements, temp tables, advisory locks). Session pooling keeps connection per client session.

31. What is 2PC vs Saga pattern?

2PC: coordinator prepares all participants then commits – atomic but blocking and not fault-tolerant across microservices. Saga: breaks distributed transaction into local transactions with compensating transactions – eventual consistency, needs idempotency and retry.

32. What is backpressure and queue design with DB?

Backpressure prevents producers overwhelming consumers. Use bounded queues, SKIP LOCKED queues, or LIMIT rate, circuit breaker, and monitor queue depth, lag, and consumer lag. Avoid unbounded table as queue without deletion/VACUUM strategy.

33. How to implement idempotency with DB?

Use idempotency key with UNIQUE constraint + INSERT ON CONFLICT DO NOTHING. Ensure operation is idempotent: same key = same result. Essential for payments, webhooks, message consumers.

34. What is Bloom filter use in DB?

Bloom filter is probabilistic structure that tells if key definitely NOT in SSTable – avoids unnecessary disk reads in LSM trees. False positive possible, false negative impossible – improves read performance.

35. What is write amplification and read amplification?

Write amplification: one logical write causes multiple physical writes due to WAL + compaction (LSM). Read amplification: one logical read reads multiple SSTables/versions. Trade-off tuned by compaction strategy and levels.

36. Row-store vs Columnar storage?

Row-store (OLTP) stores whole row together – good for point lookups and transactional writes. Columnar (OLAP) stores column together – compression, vectorized execution, good for scanning few columns over billions of rows (Parquet, Redshift).

37. What is data skew and how to handle?

Data skew when few keys have huge rows causing uneven distribution – e.g., null key or celebrity user. Handle via salting (add random prefix), broadcast small table, handle skewed keys separately, or use skewed join optimization.

38. How to implement efficient pagination?

OFFSET is bad for deep pages – scans and discards. Use keyset pagination: `WHERE id > last_id ORDER BY id LIMIT N`. Requires indexed unique sort key. For random pages use deferred join or `search_after` in Elasticsearch style.

39. What is PITR (Point-In-Time Recovery)?

PITR allows restoring DB to any timestamp using base backup + continuous WAL archiving. Essential for accidental DROP/DELETE recovery. Requires `archive_mode` on and WAL archiving to safe storage.

40. What differentiates a 10-year DB engineer from junior?

Senior makes trade-offs: they understand storage engines, MVCC, locking, cost model, not just syntax. They design for scale (sharding, partitioning), diagnose production issues (locks, bloat, replication lag), manage technical debt, choose consistency vs availability, and connect technical decisions to business impact.